2026 - 1
Intelligent Robotics

TIAGO 기반 상품 자동 적재 마트 카트 시스템 개발

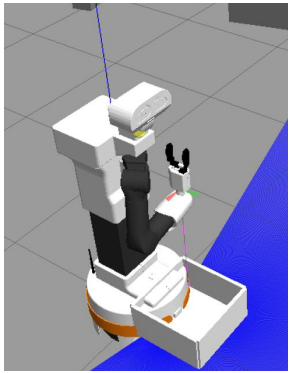
사용자 추종 및 능동적 상품 인지

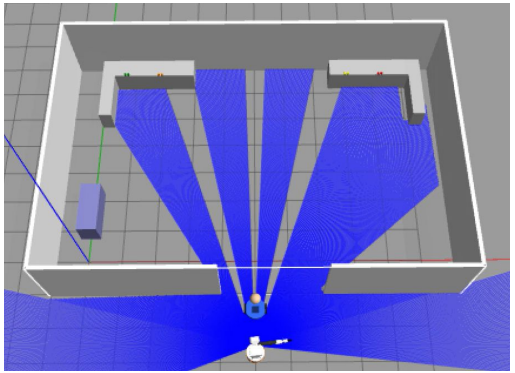
EWHA Dynamics

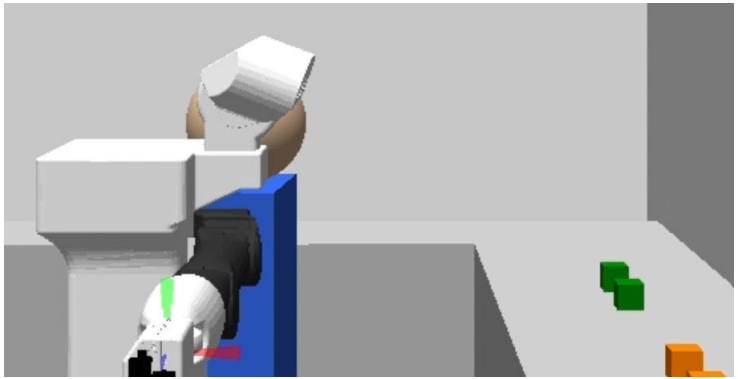
2491015 방가은
2491029 정아현

1. 프로젝트 목표 및 주제

- **주제:** TIAGO 기반 상품 자동 적재 마트 카트 시스템 개발: 사용자 추종 및 능동적 상품 인지
- **프로젝트 목표:** 대형 마트 환경에서 사용자를 안전하게 추종하고, 사용자가 지정한 물품을 로봇팔로 자동 적재하는 시스템 구축







1. 프로젝트 목표 및 주제

- 시나리오

Step 1. 사용자 추종

- 상황: 사용자가 키보드로 사람 캐릭터를 조작해 이동 시작
- 로봇 동작: 인식된 사람과의 최단 거리를 LiDAR 센서로 실시간 파악하여 정해진 간격(1m)을 유지하며 추종

Step 2. 섹션 도착 및 정지

- 상황: 사용자가 원하는 진열대 앞에서 캐릭터를 정지시킴
- 로봇 동작: 사람이 멈춘 것을 감지하고 사람과 0.45m ~ 1.0m 거리에서 정지

Step 3. 물체 지정 및 인식

- 상황: 사용자가 원하는 물건을 선택(터미널에 입력)
- 로봇 동작
    1. 로봇의 머리를 돌려 카메라가 진열대를 스캔
    2. 스캔한 물건의 실시간 3D 좌표(x, y, z)를 추출
    3. 터미널에서 물건의 이름을 사용자 input으로 받음
    4. 사용자가 원하는 물건을 집기 위한 로봇 팔의 목표 지점 계산

1. 프로젝트 목표 및 주제

Step 4. 피킹 및 바구니 적재

- 로봇 동작
  1. 계산된 경로로 로봇팔 뻗어 물체 파지
  2. 파지한 물체를 들어 올려 로봇에 설치된 바구니 위치로 이동
  3. Gripper를 열어 물건 놓기
  4. 적재 완료 후 로봇팔을 주행용 기본 자세로 복귀

Step 5. 시나리오 반복 및 종료

- 로봇 동작: 로봇의 머리를 돌려 카메라 각도를 다시 정면으로 복귀
- 반복: 사용자가 다시 이동을 시작하면 LiDAR 센서가 거리 변화를 감지하고, 사람 추종 모드로 자동 전환 후 다시 Step 1부터 시작
- 종료: 사람과 로봇이 계산대에 진입하면 모든 태스크를 완료하고 시나리오 종료

2. System Architecture - ROS Node 구성

1. **센서 기반 사람 추종 노드: `human_following.py`**
    - LiDAR 센서 데이터를 기반으로 고정 장애물(벽)을 분리하고, 사람만을 정밀하게 추종하는 노드

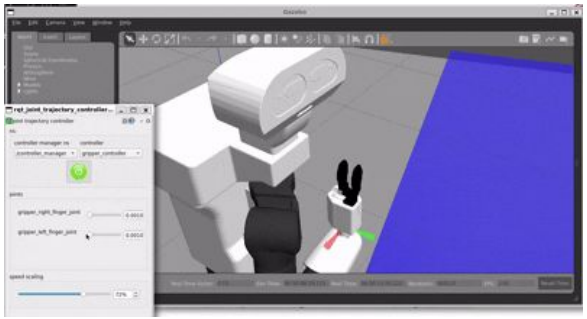
2. **로봇 정렬 노드: `shelf_docking.py`**
    - TIAGO 로봇이 물체를 잡기 편하도록 진열대를 정면으로 바라보고, 진열대 가까이 밀착하도록 하는 노드

3. **물체 인식 노드: `object_detector.py`**
    - 로봇이 상품을 감지하고, 선택할 상품을 사용자로부터 입력받은 뒤 상품의 좌표를 반환하는 노드

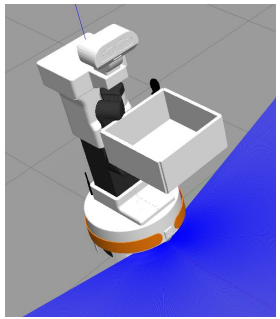
4. **물체 피킹 노드: `picking_node.py`**
    - 물체 인식 노드로부터 받은 좌표를 바탕으로 상품을 집어서 바구니 안에 넣은 뒤 다시 사람 추종을 시작하는 노드

3. 구현 방법

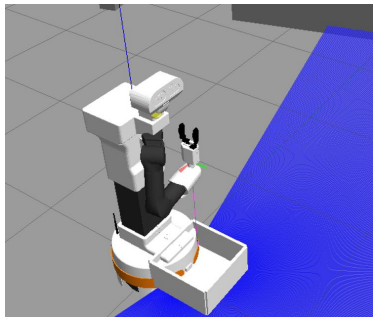
- **로봇 커스텀**
  - 기본 모델: PAL Robotics TIAGO 모델
  - 원통형 이동 베이스 위에 리프팅 몸체와 7자유도(7-DOF) 로봇팔, Gripper가 장착된 고성능 모바일 매니퓰레이터
  - TIAGO 패키지에서 집게 모양의 pal-gripper를 사용
  - 로봇의 키를 최대 길이인 145cm로 설정(**`torso.urdf.xacro`**)
  - 바구니 구현(**`tiago.urdf.xacro`**)
    - TIAGO의 하단 구동부(base_link) 앞부분을 확장하여, 가로 40cm, 세로 40cm, 높이 30cm 규격의 바구니를 추가



pal-gripper 테스트



로봇 중앙에 바구니를 단 모습
→ 로봇 팔과 겹쳐 위치 조정



최종 커스텀 결과

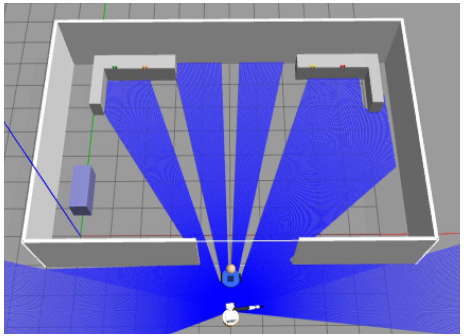
3. 구현 방법

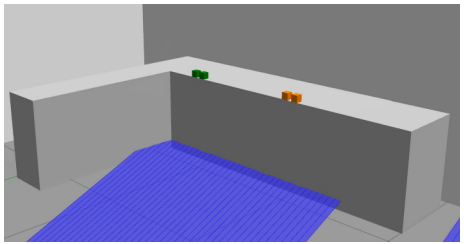
- 주요 장착 센서 및 기능

센서 유형	ROS 2 토픽명	주요 역할 및 기능
2D LiDAR	`/scan_row`	사람 추종을 위한 거리 측정, Clustering, 정면 장애물 거리 연산, 로봇 정렬 시 벽면 거리 및 각도 오차 산출
RGB-D Camera	`/head_front_camera/rgb/image_raw`	HSV 색상 마스킹을 통한 상품 색상 검출 및 위치 파악
	`/head_front_camera/depth/image_raw`	검출된 상품의 픽셀 좌표를 3차원 공간 좌표(X, Y, Z)로 변환하기 위한 거리 데이터 제공
Joint Encoder	`/joint_states`	로봇 관절 제어를 위한 위치 피드백 및 조향각 연산 지원

- **프로젝트 제안서:** RGB-D 카메라를 활용해 사람 추종과 물체 인식을 모두 수행하도록 기획.
- **수정 사항:** 사람 추종 기능은 2D LiDAR 센서로만 구현하는 것이 CPU 연산량을 줄이고 사람 형태 인식의 효율성을 높일 수 있음. 이에 RGB-D 카메라는 물체 인식 및 물체 좌표 계산에 활용하고, 사람 추종은 2D LiDAR 센서로만 구현.

3. 구현 방법





- **마트 환경 세팅**
  - 마트 벽, 과일/채소 진열대, 피킹 대상 물체(바나나, 사과, 당근, 브로콜리 - 각 2개씩)

- **구성 요소**

1. **벽 구조: 단일 모델로 통합**

   SDF에서 여러 collision/visual을 하나의 <link> 안에 묶으면 단일 Gazebo 모델로 처리

2. **진열대 구조: L자형**

   각 진열대를 하나의 static 모델로 구현. 마트 가장자리에 배치, 물체 간격 0.2m.

3. **물체 배치**

   static=false (집을 수 있음), mass=0.1kg, mu=1.0 (마찰 계수 높여 진열대에서 미끄러지지 않게), z=0.83 (진열대 상면 0.8m + 반지름 0.03m = 0.83m)

   TIAGo 그리퍼 구동 범위를 고려해, 물체 크기를 6cm cube로 통일, 물체별 색상 지정

4. **구현 과정 중 수정 사항**

   물체 위치 수정: y=6.5 → y=6.23 (카메라 인식 거리 기준 조정)

   입구 남쪽 벽 오른쪽 패널 x=3.5→4.0, 길이 5.0→4.0m로 축소

   (tuck_arm:=True 옵션이 Gazebo 구동 실패를 일으켜 사용 불가해, 팔을 편 상태로 입구를 통과해야 하는 환경 제약에 맞춰 통로 폭 조정)

3. 구현 방법

- **사람 모델**
  - 몸체(cylinder) + 상체(box) + 머리(sphere) + 바퀴 2개 + 캐스터 2개로 구성된 diff-drive 이동 가능 모델
  - /person/cmd_vel로 이동 제어, 사람 추종 노드가 해당 모델을 따라가도록 설계 / libgazebo_ros_diff_drive.so 플러그인 사용
- **구성 요소**

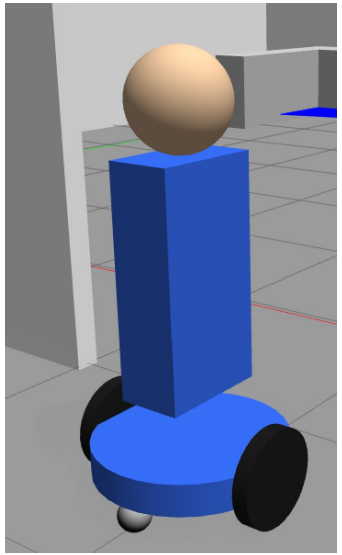
  기존 구성

  머리, 몸통, 바퀴 구조 설계 특성 상, 몸통이 높게 설계되어 무게중심이 불안정하고 작은 바퀴로 인해 넘어짐 문제 확인

  변경된 구조

  바퀴 마찰 추가 (mu=10.0), 앞뒤 캐스터 바퀴로 4점 지지, 몸통/머리는 시각적 요소만 (collision 없음)

```
person (사람 모델)
    ├── base
    │     ├── base_v (낮고 넓은 원통 베이스)
    │     ├── torso_v (몸통)
    │     └── head_v (머리)
    ├── left_wheel
    ├── right_wheel
    ├── caster (앞 균형 바퀴)
    ├── caster_back (뒤 균형 바퀴)
    ├── left_wheel_joint
    ├── right_wheel_joint
    ├── caster_joint
    └── caster_back_joint
```



3. 구현 방법 - 노드 구현

- **센서 기반 사람 추종 노드: `human_following.py`**
  - LiDAR 센서 데이터를 기반으로 고정 장애물(벽)을 분리하고, 사용자의 물리적 형체만을 정밀하게 추적하는 센서 기반 사람 추종 노드
  - 실행 결과: 로봇이 약 1m의 간격을 유지하며 사람을 따라가다가, 사람이 멈추면 사람과 0.45m ~ 1m 거리에서 멈춤.
- **주요 알고리즘 및 기능**
  1. **Density-based Clustering:**

     LiDAR 센서로 읽은 점들의 조밀도를 분석하여 밀집도가 낮은 고정된 벽면을 배제하고 움직이는 사람만 골라내서 인식.

  2. **사람 형체 식별:**

     추출된 Cluster의 너비를 계산하여, 마트의 넓은 벽이나 문틀은 제외하고 사람의 몸체 굵기와 비슷한 형태만 사람으로 정확히 판단.

  3. **Steering-linked Deceleration:**

     회전할 때 로봇이 너무 빨리 돌면 중심을 잃거나 사람과 부딪힐 수 있으므로, 회전 각도가 클수록 로봇의 속도를 자동으로 줄여 부드럽게 코너를 돌도록 함.

  4. **사람과의 거리에 따른 로봇의 대응 4단계:**

     1m 이상, 0.65m ~ 0.95m, 0.45m ~ 0.65m, 0.45m 이하로 나눠 로봇의 속도를 조절.

3. 구현 방법 - 노드 구현

- **센서 기반 사람 추종 노드: `human_following.py`**
- **어려웠던 점 및 개선 방향**

문제 상황 1: 좁은 마트 입구 통과 시, 양옆 문틀 벽이 사람보다 가깝게 인지되어 로봇이 정지하거나 벽을 바라보고 멈추는 현상 발생.

해결: 단순 최단 거리 추적 방식에서 Density-based Clustering 및 너비 계산 방식으로 전환. 문틀 벽은 포인트 개수가 과도하게 많아 벽으로 분류하여 탈락시키고, 사람 형체만 도려내어 문 통과 성공.

문제 상황 2: 사람이 갑자기 멈추거나 코너를 돌 때, 로봇이 멈추지 못하고 사람과 부딪히는 현상 발생.

해결: 사람과 너무 가까워지면(0.45m 이하) 즉시 뒤로 물러나 멈추는 '비상 제동' 기능을 추가. 또한, 회전할 때는 회전 각도에 맞춰 로봇의 속도를 자동으로 줄여주어, 관성 때문에 사람과 충돌하는 일을 방지.

3. 구현 방법 - 노드 구현

- **로봇 정렬 노드: `shelf_docking.py`**
  - TIAGO 로봇이 물체를 잡기 편하도록 진열대를 정면으로 바라보고, 진열대 가까이 밀착하도록 하는 노드.
  - 실행 시점: 사람 추종 노드 실행 완료 후
  - 실행 결과: 로봇이 진열대를 바라보는 방향으로 회전한 후, 진열대와 평행을 유지하며 진열대를 향해 가까이 이동. 진열대와 로봇 사이의 목표 간격은 0.07m. 각도 오차 0.03°, 거리 허용 오차 0.005m 내에 도달하면 '도킹 완료' 로그를 출력.

- **주요 알고리즘 및 기능**

1. **시퀀스 기반 자동 가동:**

   사람 추종 노드로부터 `START_SHELF_DOCKING` 신호를 받아 정밀 도킹 시퀀스를 시작하고, 작업 완료 시 `DOCKING_SUCCESS` 피드백을 발행하여 다음 작업으로의 유기적인 전환 유도.

2. **LiDAR 기반 진열대 평면 정렬:**

   `trimmed_mean` 알고리즘을 적용하여 센서 데이터 중 측정 오류로 인한 이상치를 제거. 최근 측정값들을 큐(Queue)에 순서대로 저장하고 평균을 산출함으로써 급격한 값의 변화를 완화하고 로봇의 매끄러운 움직임 구현.

3. **이중 제어 모드(근접 정밀 보정 모드/진입 주행 모드):**

   로봇과 진열대 간의 거리에 따라 제어 모드를 분리하여 도킹 정확도 향상.

3. 구현 방법 - 노드 구현

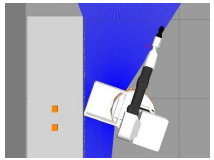
- **로봇 정렬 노드: `shelf_docking.py`**
- **어려웠던 점 및 개선 방향**

문제 상황 1: 초기에는 사람 추종 노드 안에 로봇 정렬 기능까지 구현하려고 함. 하지만 LiDAR 센서가 Density-based Clustering으로 사람과 진열대를 구분하지 못해 로봇이 진열대 쪽으로 이동하지 않음.

해결: 사람 추종 노드와 로봇 정렬 노드를 분리함. 사람 추종 노드에서는 사람을 따라 정지하는 것까지만 구현하고, 로봇 정렬 노드에서 진열대를 향해 로봇이 밀착 정렬하는 기능을 구현함.

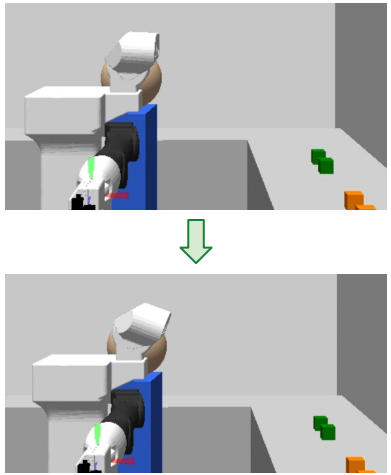
문제 상황 2: 진열대에 가까이 이동한 후에 진열대와 평행하도록 각도를 조정하다가 진열대에 닿아서 멈춰 버림.

해결: 거리가 목표에 근접하면 전진을 멈추고 제자리 회전으로만 각도 보정.



3. 구현 방법 - 노드 구현

- **물체 인식 노드: `object_detector.py`**
  - 실행 시점: 로봇 정렬 노드 실행 완료 후
  - 실행 결과: 로봇이 고개를 숙여서 상품을 감지. 로봇의 사야에 보인 상품 리스트와, 피킹 가능 반경 내에서 인식된 상품 리스트를 각각 출력. 피킹 가능 반경 내에서 감지된 상품이 있다면 리스트로 출력 후 사용자에게 상품 종류와 수량을 입력받음. 감지된 상품이 없다면 다시 사람을 바라보고 사람 추종 모드로 전환.



```
[INFO] [1780763768.488691709] [object_detector]: 물체 인식 및 도달 가능 영역 연산 노드 구동 시작
[INFO] [1780763769.430244443] [object_detector]: 자동 진열대 스캔 시퀀스 가동
[INFO] [1780763769.433195688] [object_detector]: 카메라 이미지 최신화 대기 중...
[INFO] [1780763772.438152663] [object_detector]: 이미지 수신 확인됨, 스캔 시작
======================================================================
전체 진열대 물체 스캔 결과 (원거리 포함)
======================================================================
- carrot: 총 2개 감지 완료
 ======================================================================
피킹 가능 상품 리스트
======================================================================
- carrot: 피킹 가능 수량 2개
======================================================================
어떤 상품을 선택하시겠습니까? (종료: q): carrot
해당 상품을 카트에 몇 개 담으시겠습니까? (1 ~ 2개 가능): 2
[INFO] [1780763778.925009523] [object_detector]: [carrot] 지정 수량 2개 연산 및 MoveIt 시퀀스 토픽 방출
[INFO] [1780763778.929558225] [object_detector]: [carrot_1] 추출 좌표 전송 -> X:-0.13m, Y:0.25m, Z:0.84m
[INFO] [1780763779.939541150] [object_detector]: [carrot_2] 추출 좌표 전송 -> X:0.07m, Y:0.25m, Z:0.86m
[INFO] [1780763780.945226764] [object_detector]: 피킹 모드로 전환합니다
```

3. 구현 방법 - 노드 구현

- 물체 인식 노드: `object_detector.py`
- 주요 알고리즘 및 기능
  1. **OpenCV HSV 필터링 기반 물체 인식:**

     입력된 RGB 이미지 데이터를 HSV로 변환한 뒤, 각 상품 고유의 색상 범위만 남기고 나머지는 모두 검은색으로 지우는 마스킹을 수행. 마스킹된 이미지에서 윤곽선(`findContours`) 및 모멘트(`moments`) 연산을 통해 노이즈 면적을 제거하고 각 상품의 중심점 픽셀 좌표(`cx`, `cy`) 추출.

  2. **물체의 3차원 좌표 연산:**

     검출된 픽셀 좌표와 해당 지점의 깊이 데이터(Depth)를 매핑하고, 카메라의 초점거리 및 중심점 파라미터를 활용한 Inverse Projection 수식 적용.

     이렇게 구한 3차원 좌표(X, Y, Z)를 `PoseStamped` 토픽으로 내보냄.

  3. **피킹 가능 범위 내 상품 리스트 출력:**

     TIAGo 매니퓰레이터가 물체를 피킹할 수 있는 거리를 고려하여, depth 값이 0.5m ~ 1.0m 범위에 존재하는 상품 데이터만 선별 및 별도의 리스트로 출력.

  4. **카메라 고개 숙이기 및 촬영 대기:**

     로봇의 머리 관절을 아래로 숙여 정면에 있던 카메라가 진열대 하단 상품들을 바라보도록 시선 변경. 고개가 완전히 고정되고 최신 이미지가 들어올 때까지

     일정 시간 대기하는 안정화 로직 구현.

3. 구현 방법 - 노드 구현

- **물체 인식 노드: `object_detector.py`**
- **어려웠던 점 및 개선 방향**

문제 상황 1: 로봇이 고개를 숙이는 도중에 `time.sleep(2.5)` 시간이 끝나서 좌표 연산을 시작해 버림. 물체 인식을 2번 이상 반복해야
충분히 고개를 숙인 상태에서 물체 인식과 좌표 연산을 시작함.

해결: `time.sleep`을 0.5로 늘림

3. 구현 방법 - 노드 구현

- 물체 피킹 노드 : **`picking_node.py`**
  - 물체 인식 노드로부터 받은 좌표를 바탕으로 상품을 집어서 바구니 안에 넣은 뒤 다시 사람 추종을 시작하는 노드

- 전체 파이프라인
  1. **/target_object_pose 수신 → TF 변환(로봇 기준 좌표로 변환) → 큐 적재**

  2. **피킹 루프**
     [그리퍼 열기 → 어프로치 → 하강 → 그리퍼 닫기 → 상승 → 드롭 → 팔 복귀] × n개
     *n = 물체 개수

  3. **머리 정면 복귀 → 회전 탐색**

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 주요 알고리즘 및 기능

1. **좌표 수신 및 변환**

   TF2 라이브러리를 이용해 카메라 좌표 → 로봇 기준 좌표로 자동 변환

   타임스탬프를 초기화(stamp = Time())로 시뮬레이션 시간과 실제 시간 차이로 인한 변환 실패 방지

2. **큐 기반 다중 물체 처리**

   물체 좌표가 들어올 때마다 큐(deque)에 쌓아두고, 마지막 수신 3초 후에 한꺼번에 처리를 시작

   for 루프로 하나씩 순서대로 집고 놓기를 반복 (1개로 진행, 코드 구조상 2개까지 쉽게 확장 가능)

3. **2단계 파지 접근**

   1) 어프로치: 물체 위 0.4m 지점까지 자유 경로로 이동 (OMPL 경로 탐색) - 진열대 충돌 박스 상단에서 시작
   2) 하강: Cartesian Path - 그 지점에서 물체까지 1cm 단위 직선으로 수직 하강

   * 파지 Z 좌표: 카메라가 추정한 값이 매번 흔들려서 사용하지 않고, 진열대 높이에서 계산한 world 고정값 0.84m를 사용

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 주요 알고리즘 및 기능

4. MoveIt

Planning Scene: 경로 계획 시 진열대를 피해야 할 장애물로 인식하도록 가상 박스 등록

타임스탬프를 초기화(stamp = Time())로 시뮬레이션 시간과 실제 시간 차이로 인한 변환 실패 방지

1) 파지 직전

물체(0.83m)가 박스 안에 들어가도록 일부러 실제(0.8m)보다 높게(1.0m) 설정 → OMPL이 z<1.0 구간을 막힌 것으로 인식

2) 어프로치 : 충돌 검사 ON

목표: 물체 위 0.4m = z 1.24m (박스 위) → 반드시 z > 1.0 위에서 내려오는 경로를 선택

3) 하강 : 충돌 검사 OFF

박스 안이 목표지점(z = 0.84m)이라 충돌 검사 켜면 MoveIt이 거부 → OFF로 전환

z 1.24m에서 z 0.84m까지 수직 직선으로만 내려옴

4) 상승

(충돌 OFF): z 0.84m → 1.24m → 박스 안에서 위로 탈출

상승 후 ON: 충돌 박스 제거 → 드롭, 복귀 경로 단순화

상승 속도: 0.05로 제한(빠른 가속으로 물체 낙하 방지)

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 주요 알고리즘 및 기능

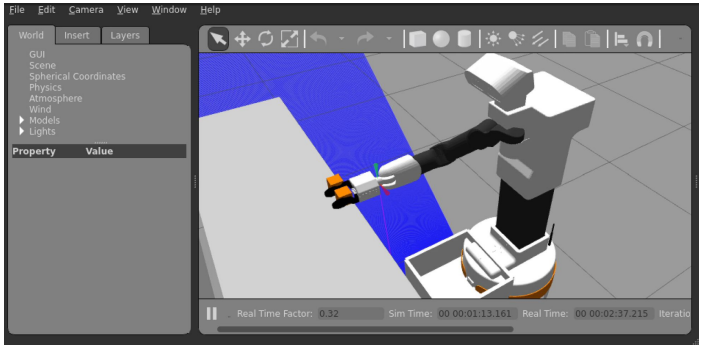
5. **로봇 베이스 잠금 스레드**

   피킹 루프 실행 중 별도 스레드에서 0.2초마다 정지 명령 Twist() 발행

   피킹 구현을 위해 단독 테스트할 때 로봇이 외력에 밀려 움직이는 것을 방지하기 위한 설계

6. **피킹, 바구니 적재 완료 후**

   머리를 정면으로 돌리고, 10초간 제자리 회전하며 사람을 탐색한 뒤 사람 추종 모드로 전환



3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

1. **pymoveit2 크래시**

   pymoveit2의 plan() 함수가 내부적으로 ROS2 실행기를 자동 생성하려 하는데, WSL2 ROS2 Humble 환경에서 이 과정이 지원되지 않아 크래시 발생

   코드 실행 시점에 문제 함수(rclpy.spin_once)를 time.sleep으로 덮어씌우는 방식으로 간단하게 우회

2. **Multi Thread 데드락**

   MoveIt이 팔을 움직이는 동안 로봇의 다른 토픽(좌표 수신 등)이 응답하지 않는 문제 발생

   - 기본 실행기(SingleThreadedExecutor)는 한 번에 하나의 작업만 처리할 수 있어서, MoveIt 동작이 끝날 때까지 다른 콜백이 실행되지 않음을 확인

   4개 스레드를 동시에 돌리는 MultiThreadedExecutor와 여러 콜백이 겹쳐 실행될 수 있는 ReentrantCallbackGroup을 적용

* 크래시(Crash)
프로그램이 예상치 못한 오류로 갑자기 종료되는 것(피킹 시작 버튼을 누르는 순간 스레드 하나가 오류 발생)

*데드락(Deadlock)
두 작업이 서로 상대방이 끝나기를 기다리다가 둘 다 멈춰버리는 상황 (MoveIt이 팔을 움직이는 동안 단일 스레드가 그 작업만 잡고 있어서 다른 토픽 수신이 아예 실행되지 못함)

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

3. **접근 경로가 물체를 충격**

   물체를 정면 수평으로 잡으려 하면 그리퍼가 진열대 앞면을 통과해야 해서 불가능

   위에서 내려오는(top-down) 방식 선택

   이후, 그리퍼가 물체에 비스듬히 접근해 물체와 충돌하는 문제 발생

   가정1. 그리퍼 방향을 정의하는 쿼터니언 값이 문제라고 가정 후 손목 구성이 다른 세 후보 테스트

   A [0, 1, 0, 0] → 팔이 측면에서 접근해 실패

   B [0.707, 0.707, 0, 0] → 물체를 옆으로 밀어내는 동작 발생

   C [0.707, -0.707, 0, 0] → 진열대 위로 접근 경로 자체가 막힘

   셋 다 동일 증상 → 쿼터니언이 원인 아님을 확인

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

3. **접근 경로가 물체를 충격**

가정2. 어프로치 목표점 수정(APPROACH_OFFSET_Z)

기존) 어프로치 목표점 = 물체 z(0.83) + 오프셋(0.15) = z 0.98m / 충돌 박스 상단 = z 1.0m

-> OMPL: z < 1.0 접근을 선택하지 않으므로, 0.98에 도달하기 위해 비스듬한 경로를 선택했음을 확인

[1,0,0,0] 유지 + APPROACH_OFFSET_Z = 0.4로 수정

어프로치 목표점 = 0.83 + 0.4 = 1.23m

충돌 박스(1.0m) 위가 되어, OMPL이 반드시 위에서 내려오는 경로를 선택

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

4. 파지 z 높이 조정

카메라가 추정한 Z값(pz)에 오프셋(GRASP_Z_OFFSET)을 더해 파지 높이를 조정하는 방식을 처음에 시도

1) 이진 탐색 시도

GRASP_Z_OFFSET = 0.07 → 물체 위에 뜸 (안 닿음)

GRASP_Z_OFFSET = 0.04 → 여전히 안 닿음

GRASP_Z_OFFSET = -0.05 → 진열대 통과 (너무 낮음)

GRASP_Z_OFFSET = -0.03 → 여전히 낮음

-> 탐색 중 pz 값 자체가 매번 다르게 들어오는 것을 확인

2) 진열대 높이에서 직접 계산한 고정값(GRASP_Z_FIXED) 사용 방식으로 전환

진열대 상면 0.80m + 물체 반지름 0.03m + 접촉 여유 0.01m = 0.84m

3. 구현 방법 - 노드 구현

- 물체 인식 노드: `picking_node.py`
- 어려웠던 점 및 개선 방향

5. **그리퍼 닫힘 시 물체 튕김**

   그리퍼가 닫히면서 물체가 진열대 밖으로 튕겨 나가는 문제 발생

   원인1. 속도

   duration_sec=2: 빠르게 닫히면서 접촉 순간 충격력이 커 물체가 진열대 밖으로 튕겨 나감

   원인2. 대기 시간

   시뮬레이터가 실제의 0.3배 속도로 돌아가는데(RTF≈0.3), 시뮬 2초는 실제로 6초 이상 걸림을 확인

   time.sleep(2.3초)로는 그리퍼가 다 닫히기 전에 다음 단계로 넘어감

   해결

   duration_sec=5로 느리게 닫기 + time.sleep(5 * 2.5 + 0.5) = 13초 대기로 RTF 보정

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

6. **DROP 플래닝 실패**

드롭 쿼터니언: [1,0,0,0] - 파지 자세 그대로 유지(파지한 상태에서 팔 방향 변경 없이 위치만 이동하게 설정)

문제: 물체를 집은 뒤 바구니 위(x=0.35, y=0.0)로 팔을 이동할 때 처음 설정한 z=0.3에서 플래닝 실패
파지 후 상승한 위치(z 1.24m)에서 바구니 근처 낮은 위치까지 내려가는 경로가 팔 관절 한계에 걸렸기 때문이라고 판단

z = 0.30, 0.50, 0.60, 0.70 테스트

해결
DROP_POSITION = [0.35, 0.0, 0.70] : 0.7 높이에서 물체가 바구니 중심으로 낙하

3. 구현 방법 - 노드 구현

- 물체 인식 노드: **`picking_node.py`**
- 어려웠던 점 및 개선 방향

7. **robot_spawn.launch.py - 안정적 피킹을 위한 로봇 스폰 위치 고정**

   매번 사람 추종 → 도킹 과정을 거치면 도킹 위치가 달라져 파지 실패 원인이 코드인지 위치인지 구분 불가함을 확인

   해결

   위치 변수를 제거하기 위해 로봇을 고정 좌표에 스폰

   로봇 spawn 위치: x=1.55, y=5.45, yaw=π/2 (world +Y 방향을 향함)

   대상 물체(carrot1) 가 로봇 정면에 오도록 조정: world 좌표 (1.8, 6.5)

3. 구현 방법

- 물체 인식 노드: `picking_node.py`
- 어려웠던 점 및 개선 방향

8. **gripper.gazebo.xacro - 그리퍼 접촉 물리 파라미터 조정**

kp: 손가락이 물체 안으로 얼마나 파고드느냐에 따라 힘을 결정하는 계수

값이 크면 조금만 닿아도 강하게 밀어내고, 작으면 조금 파고들어도 부드럽게 반응

기본값(1,000,000)에서 50,000 → 5,000 → 400까지 낮춰가며 조정

minDepth: 접촉을 인식하기 시작하는 최소 침투 깊이

처음 0.1(10mm)로 설정했다가 오히려 접촉이 늦게 시작되는 문제가 있어 0.0으로 제거

mu1, mu2(마찰계수): 손가락과 물체 사이 마찰

1.0으로 설정해 물체가 미끄러지지 않도록 함

kd: 접촉 시 발생하는 진동/튕김을 억제하는 감쇠(damping) 값

3. 구현 방법

- 물체 인식 노드: `picking_node.py`
- 어려웠던 점 및 개선 방향

8. **gripper.gazebo.xacro - 그리퍼 접촉 물리 파라미터 조정**

Gazebo Classic ODE 물리 엔진: 손가락이 물체 표면에 minDepth 이상 파고들어야만 접촉으로 인식

접촉 인식 조건: 실제 침투 깊이 ≥ minDepth

최대 침투 가능 깊이: effort / kp

문제 상황: PAL 원본값 (kp=1,000,000 / minDepth=0.1m / effort=16N)

최대 침투 가능 깊이 = 16 / 1,000,000 = 0.016mm

필요 깊이 (minDepth) = 100mm

→ 0.016mm << 100mm → 접촉 미인식 → 마찰력 0

→ 필요한 힘: 1,000,000 × 0.1 = 100,000N → 물리적으로 불가능을 확인

→ PAL 원본값은 파지가 불가능한 조합임을 확인. 실제 하드웨어용으로 맞춰진 값이거나, 시뮬레이션 파지 자체를 염두에 두지 않은 기본값으로 추정

3. 구현 방법

- 물체 인식 노드: `picking_node.py`
- 어려웠던 점 및 개선 방향

8. **gripper.gazebo.xacro - 그리퍼 접촉 물리 파라미터 조정**

해결

effort ≥ kp × minDepth 조건을 만족하는 값으로 조정 / PAL Robotics 레포 권한 문제로 로컬에서만 진행

단계	kp	minDepth	비고
PAL 원본	1,000,000	0.1m(100mm)	필요 힘 100,000N → 파지 원천 불가
1차 수정	50,000	0.01m(10mm)	최대 침투 0.32mm << 10mm → 여전히 미인식
2차 수정	5,000	0.01m(10mm)	최대 침투 3.2mm >> 10mm 아직 부족
3차 수정	400	0.0	minDepth=0 (침투 즉시 인식) → 여전히 contacts=0

(추가) kd=10: 감쇠를 높여 접촉 시 안정성 향상

3. 구현 방법

- 물체 인식 노드: `picking_node.py`
- 어려웠던 점 및 개선 방향

9. object_detector.py 수정

1) 고개(head_2_joint) 이동 완료를 시간 대기(sleep) 대신 /joint_states 폴링으로 확인하도록 변경
/joint_states 토픽을 주기적으로 읽어, head_2_joint가 실제로 목표 각도에 도달했는지 확인한 뒤 스캔을 시작하도록 변경
→ Gazebo 속도가 느릴 때 고개가 목표 각도에 도달하기 전 스캔하는 문제 해결

2) TF 프레임명을 실제 발행되는 이름(color_optical_frame)으로 수정
object_detector 작성 시점에는 확인할 수 없었음 / 시뮬레이션을 실행 중 TF 변환 실패 에러 발견 후 실제 발행 프레임명(color_optical_frame)으로 수

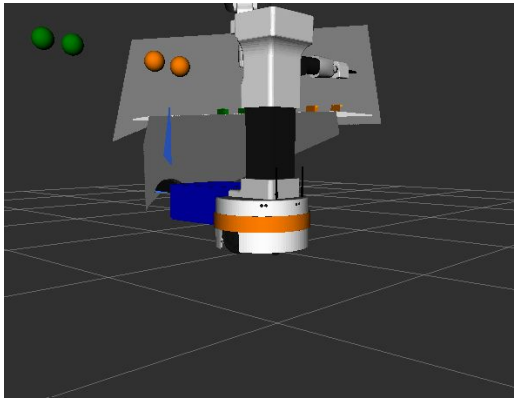
3) depth 유효 범위 하한, tilt 목표값 조정 (물체 인식 안정화)

4. 결과 및 데모

5. 향후 개선 및 고도화 계획

1. RViz 좌표 변환 오류

- **문제 상황:** 프로젝트 제안서에서는 상품을 인식한 뒤에 RViz Marker로 상품 위에 번호표를 띄우도록 구상하였음. 하지만 구현해보니
  로봇이 물체 좌표 자체는 정확히 반환하고 있음에도 RViz 상에서는 물체 바로 위가 아닌 허공에 마커가 표시됨.



- **원인 분석:** 로봇이 계산한 실제 상품의 좌표는 카메라 렌즈 중심을 기준으로 구한 값인 반면, RViz Marker 생성 및 발행 함수에서 정의한 수식은 로봇의 바닥면(`base_footprint`)을 기준으로 임의의 숫자로 하드코딩했음. 이때 하드코딩된 값이 로봇의 키 정보나 실제 위치와 맞지 않아 허공에 마커가 그려짐.

- **조치:** RViz 좌표 변환을 위한 적절한 값을 찾는 데 시간이 오래 걸릴 것으로 예상되어 RViz Maker 구현은 후순위로 두고, 각 노드 구현을 먼저 마침.

- **보완 및 디버깅 계획:** 하드코딩한 값을 정확한 로봇 정보로 수정하거나 ROS 2의 표준 공간 변환 시스템인 TF 라이브러리를 통해 카메라 좌표계를 월드 좌표계로 자동 변환.

5. 향후 개선 및 고도화 계획

2. 피킹 반경 내 물체 미감지 시 회전 중단

- **문제 상황:** 피킹 가능 범위 내 상품 미존재 시, 40cm 후진 후 10초간 저속 회전하며 사용자를 재탐색하도록 설계된 예외 처리 로직이 작동됨. 그러나 실행 과정에서 목표한 10초를 채우지 못하고 회전 제어 명령이 조기에 중단되는 에러가 발생함. 로봇이 충분히 회전하지 못한 상태에서 주행 스크립트가 강제 종료되며 '사람 추종 모드로 전환합니다' 라는 로그를 조기 출력함.
- **원인 분석:** 시스템 내부 스레드 간 충돌 또는 타임아웃이 원인으로 추정됨.
- **조치:** 로봇이 사람을 감지할 수 있는 시야각을 확보할 때까지 해당 객체 인식 노드를 터미널에서 연속적으로 수동 재실행하여 누적 회전 각도를 확보함.
- **보완 및 디버깅 계획:** 파이썬의 `time.sleep()` 대신 ROS 2 노드 자체 타이머(`self.create_timer`)를 도입하여 10초 동안 안정적으로 회전 명령이 끊김 없이 퍼블리시되도록 수정.

5. 향후 개선 및 고도화 계획

3. 그리퍼가 물체를 통과하는 문제

1) 그리퍼가 닫힐 때 Gazebo 물리 엔진에서 손가락과 물체 사이의 접촉이 전혀 감지되지 않(gz topic -e /gazebo/default/physics/contacts에서 빈 출력)
2) 손가락 collision box가 물체와 겹쳐야 하는 위치에 있음에도 통과

조치1. effort limit 진단 (16N → 50N)

그리퍼에 실제로 힘이 걸리는지 확인하려 했으나, Gazebo Classic에서 position control 조인트는 /joint_states의 effort 값이 항상 0으로 보고됨을 확인(정상 동작)

→ 실제 적용 힘을 직접 확인할 수 없어, 16N→50N으로 올려 테스트했으나 동일 결과 확인(effort 부족이 원인이 아님을 확인)

조치2. 접촉 물리 파라미터 조정(kp, minDepth)

→ PAL 원본값이 파지가 불가능한 조합임을 확인 후, 단계적으로 값들을 조정했지만 접촉 감지를 확인할 수 없었음

5. 향후 개선 및 고도화 계획

3. 그리퍼가 물체를 통과하는 문제

원인 분석 - Gazebo 물리 처리 순서 분석

1단계: 충돌 감지 (broadphase/narrowphase)

→ collision box끼리 기하학적으로 겹치는지 판단

2단계: 접촉 반응 (contact response)

→ kp, minDepth, mu 등이 작동하는 구간

→ 얼마나 튕겨낼지, 마찰력 얼마나 줄지 계산

보완 및 디버깅 계획

파라미터 문제가 아닌 손가락 collision box의 실제 TF 위치가 물체 위치와 어긋날 가능성 추정

→ 실제 로봇 손가락(그리퍼) 위치를 측정해 물체 좌표와 비교

5. 향후 개선 및 고도화 계획

4. OMPL - 경로 무작위성

MoveIt의 OMPL 플래너: 무작위 샘플링 기반, 동일한 목표 좌표에서도 실행마다 다른 경로를 선택함.

(PAL Robotics 공식 문서: "at each execution the selected plan will differ due to the random nature of the motion planners")

개선 방향

home → 고정 pre-grasp 관절 자세(joint space) → cartesian 하강 순서로

중간 경유점을 관절 공간에서 고정하면 OMPL 탐색 공간이 좁아져 경로 안정성을 높이는 방향

*출처: https://wiki.ros.org/Robots/TIAGo/Tutorials/MoveIt/Pick_place

5. 향후 개선 및 고도화 계획

5. 중앙 제어 노드 구현 시도

- **문제 상황:** 사람 추종, 로봇 정렬, 물체 인식, 물체 피킹 등 개별 노드들을 유기적으로 연결하여 전체 과정을 자동화하기 위해 중앙 제어 노드(`sequence_manager.py`) 구축 및 `/sequence_control` 토픽 연동을 시도함. 그러나 전체 시나리오 가동 중 노드 간 제어권 전환 및 활성화/비활성화 과정이 매끄럽지 못해 흐름이 끊기거나 고착되는 현상이 발생함.
- **원인 분석:** 개별 노드가 작업을 완료한 후 발행하는 '완료 로그'가 실행 환경의 센서 노이즈나 네트워크 지연에 따라 간헐적으로 수신되지 않거나, 무한 대기 상태에 빠지면서 종료 신호가 중앙 제어 노드에 전달되지 않는 현상이 발생함. 작업 완료 시점을 명확히 인지하지 못하는 상황에서 중앙 제어 노드가 강제로 다음 시퀀스를 실행하거나 노드를 재시작하려 함에 따라 시스템 전체의 실행 흐름이 꼬이게 됨.
- **조치:** 중안 제어 노드 구현은 보류, 직접 노드를 실행 및 제어하여 전체 시나리오의 성공률 확보.
- **보완 및 디버깅 계획:** 외부의 강제 제어 방식이 아닌, ROS 2 표준 Lifecycle 구조를 통해 노드의 상태(Unconfigured, Inactive, Active)를 명확히 관리. 이를 통해 중앙 제어 노드가 각 노드의 작상태를 실시간으로 확인하고, 제어권을 안전하게 넘겨주어 시스템 오작동을 방지. 추가로, 로봇 정렬 노드의 센서 노이즈나 무한 대기 상태에 대비한 예외 처리를 강화하여 시스템의 안정성을 근본적으로 확보.